# Python Engineering - Class 05 Theory Guide

Listas, tuplas y trayectorias

Esta guía explica la teoría de programación, matemática y/o ingeniería que da sentido al ejercicio. El objetivo es comprender el modelo antes de memorizar sintaxis.

## 1. Por qué necesitamos colecciones

Una simulación produce una secuencia de estados, no un único valor.

### Relación / ecuación

```
trajectory=[state_0,...,state_N]
```

### Conexión con el código

Las listas permiten crecer dinámicamente cuando el número de pasos no se conoce de antemano.

## 2. Snapshots de estado

Cada tuple guarda los valores correspondientes a un instante concreto.

### Relación / ecuación

```
state_k=(h_k,v_k,t_k)
```

### Conexión con el código

Añadir una tuple nueva en cada iteración conserva el histórico de la trayectoria.

## 3. Mutabilidad y referencias

Las variables Python referencian objetos. Modificar un objeto mutable se observa desde cualquier nombre que apunte al mismo objeto.

### Relación / ecuación

```
a=b -> mismo objeto
```

### Conexión con el código

Por eso una tuple que contiene una list no convierte la list interna en inmutable.

## 4. Extracción de datos

Una vez almacenada la trayectoria, las magnitudes se pueden derivar de la colección.

### Relación / ecuación

```
max_h=max(state[0]); v_final=trajectory[-1][1]
```

### Conexión con el código

Esto prepara el paso posterior a arrays NumPy y procesamiento vectorizado.

## Método manual de validación

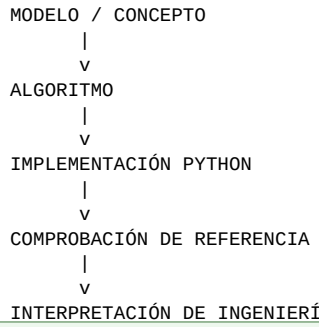
Para cualquier ejercicio de esta clase, utiliza este protocolo antes de confiar en la salida:

- 1. Escribe los valores de entrada con unidades o tipos.
- 2. Predice signo, escala, tipo y shape del resultado.
- 3. Calcula a mano un caso pequeño cuando sea posible.
- 4. Ejecuta el código y compara con la predicción.
- 5. Cambia un input de forma controlada y explica el efecto.

## Conexión con el resto del curso

Las colecciones de trayectoria evolucionarán hacia matrices NumPy, archivos, gráficas y pipelines de datos.

## Resumen final




La sintaxis es la herramienta. La capacidad transferible es saber modelar, estructurar, validar e interpretar.